Descrição dos entregáveis à disciplina de Design Profissional (Profª Isabella Luiza Souza) em parceria com a disciplina de Desenvolvimento Front-End Para Web (Profª Aline Santos).

Entregável I - Escrita de um README.md (com descrição do projeto, público-alvo, funcionalidades, tecnologias utilizadas, requisitos mínimos, instruções para execução e desenvolvedores) do sistema disponibilizado pela professora Aline.

Entregável II - Relatório de Correção de Software (RCS) simplificado do sistema disponibilizado pela professora Aline.

Desenvolvedores

- Nome: Melissa Chagas Lameiras
- RGM: 04896622-3

Instruções gerais

1. Utilize este documento como modelo para a elaboração do Relatório de Correção de Software (RCS);
2. Mantenha a formatação original do arquivo, preservando títulos, subtítulos, tabelas e demais elementos estruturais do documento;
3. Substitua todos os textos entre [colchetes] pelas informações do seu grupo;
4. Após preencher cada campo, remova os colchetes, mantendo apenas o conteúdo inserido;
5. A quantidade de bugs documentados não é fixa. As tabelas e seções destinadas ao registro dos problemas poderão ser ampliadas ou reduzidas conforme a quantidade de erros identificados e corrigidos pela equipe;
6. O documento deverá ser preenchido de forma clara, técnica e organizada, registrando as principais etapas de identificação, análise, correção e validação dos problemas encontrados;
7. Após a finalização, o relatório deverá ser exportado e salvo em formato PDF;
8. O arquivo PDF gerado deverá ser incluído e entregue juntamente com os demais artefatos do projeto por meio do repositório no GITHUB;
9. Antes da entrega, revise o documento para verificar se todos os campos obrigatórios foram preenchidos, se não restaram textos de exemplo ou marcações entre colchetes e se as informações apresentadas correspondem às correções efetivamente realizadas no sistema.

Relatório de Correção de Software (RCS)

O Relatório de Correção de Software (RCS) é um documento técnico utilizado para registrar os problemas identificados em um sistema, as análises realizadas, as correções implementadas e os testes executados para validar seu funcionamento. Muito presente nas áreas de Engenharia de Software e Garantia da Qualidade (QA), esse documento também pode ser encontrado com outras denominações, como Relatório de Manutenção Corretiva, Registro de Correção de Defeitos (Bug Fix Report), Documento de Correção de Bugs ou Relatório de Atualização de Software. Seu principal objetivo é garantir a rastreabilidade das alterações realizadas e documentar formalmente o processo de correção do sistema.

1. Identificação do projeto

Sistema analisado

Career Dreamer — Mapeamento Vocacional

Objetivo do sistema

Página web em que estudantes informam nome, hobbies e áreas de interesse. Os dados são enviados a uma API com inteligência artificial, que devolve um perfil profissional em JSON. A página de resultado exibe as carreiras em tecnologia mais compatíveis, com porcentagem de compatibilidade, competências-chave e roteiro de estudos.

Tecnologias utilizadas

HTML5, CSS3 e JavaScript (ES6) puro, sem frameworks. Comunicação com a API REST por fetch, em JSON, e uso do SessionStorage para levar o resultado do formulário à página de resultado.

2. Levantamento de problemas encontrados

ID	Categoria	Descrição do problema	Severidade
BUG-01	Funcionalidade	Botão "Descobrir minha carreira" não faz nada ao ser clicado (issue #01).	Alta
BUG-02	Funcionalidade	Campo Nome só aceita números e pode ser enviado	Alta

		vazio ou com uma letra (issue #02).	
BUG-03	Interface	Campos do formulário lado a lado, espremidos, com botão gigante ao lado (issue #03).	Alta
BUG-04	Responsividade	No celular a página aparece minúscula e com rolagem horizontal (issue #04).	Alta
BUG-05	Interface	Navegador oferece traduzir a página do inglês; leitor de tela lê com sotaque errado (issue #05).	Média
BUG-06	Interface	Erros de português: "Vocasional" e "interêsses" (issue #06).	Média
BUG-07	Interface	Rótulos dos campos quase brancos sobre fundo branco, contraste insuficiente (issue #07).	Média
BUG-08	Funcionalidade	Clicar no rótulo não leva ao campo e o foco do teclado (Tab) não é visível (issue #08).	Média
BUG-09	Interface	Página sem estrutura semântica (sem h1, regiões nem lista), apontada pelo Lighthouse (issue #09).	Média
BUG-10	Funcionalidade	Formulário aceita envio vazio ou só com vírgulas e gera itens vazios (issue #10).	Alta
BUG-11	Interface	Aviso feio do navegador (alert) no lugar de uma janela do site (issue #11).	Média
BUG-12	Funcionalidade	Formulário não envia os dados à API nem trata carregamento, erro e demora (issue #12).	Alta
BUG-13	Funcionalidade	Página de resultado abre em branco (issue #13).	Alta

Descrição das categorias

- Interface: problemas visuais, alinhamento, cores, fontes e organização da página.
- Responsividade: falhas de adaptação em diferentes tamanhos de tela.

- Funcionalidade: botões, menus, formulários ou recursos que não funcionam corretamente.

Descrição dos níveis de severidade

- Baixa: O problema não impede o uso da funcionalidade e possui impacto apenas estético ou de pouca relevância para a experiência do usuário. Exemplos: desalinhamento de elementos, diferenças de cores, fontes inconsistentes ou pequenos problemas visuais.
- Média: O problema afeta a usabilidade ou dificulta a execução de uma tarefa, mas ainda permite que o usuário utilize o sistema por caminhos alternativos. Exemplos: mensagens de erro incorretas, problemas de responsividade em algumas telas ou dificuldades de navegação.
- Alta: O problema impede o funcionamento correto de uma funcionalidade importante do sistema, comprometendo significativamente a experiência do usuário. Exemplos: botões que não executam ações, formulários que não enviam dados ou menus que não funcionam.
- Crítica: O problema torna uma funcionalidade essencial inutilizável ou impede a utilização de parte significativa do sistema. Exemplos: impossibilidade de acessar páginas importantes, falhas que impedem o cadastro ou o login, ou erros que tornam a aplicação inutilizável em determinados cenários.

3. Análise e correção

BUG-01

- Descrição do problema: Ao preencher os campos e clicar em "Descobrir minha carreira", nada acontecia: nenhuma mensagem e nenhuma mudança na tela (issue #01).
- Causa identificada: O botão estava declarado com type="button", que não envia o formulário. Como o app.js trata o evento submit, o código de envio nunca era executado.
- Correção realizada: Troca para type="submit": o clique (ou a tecla Enter nos campos) passa a disparar a validação nativa do HTML5 e o evento submit.
- Arquivos alterados:
    - index.html

BUG-02

- Descrição do problema: O campo Nome só aceitava números (e abria o teclado numérico no celular) e podia ser enviado vazio ou com uma única letra (issue #02).

- Causa identificada: O input estava com type="number" e não tinha required, minlength nem maxlength.
- Correção realizada: O input passou a type="text" com required, minlength="2" e maxlength="60". No app.js o nome é limpo com trim() e revalidado (mínimo de 2 caracteres), evitando nomes formados só por espaços.
- Arquivos alterados:
    - index.html
    - js/app.js

BUG-03

- Descrição do problema: Os campos apareciam um ao lado do outro, apertados, e o botão ficava grande e redondo na lateral, impedindo a leitura do que era digitado (issue #03).
- Causa identificada: A classe .formulário usava display: flex com flex-direction: row, o que dispõe os filhos em linha.
- Correção realizada: Troca para flex-direction: column. Em coluna, os itens do contêiner flex se esticam na largura, de modo que os campos ficam um embaixo do outro e o botão ocupa toda a largura do cartão.
- Arquivos alterados:
    - css/style.css

BUG-04

- Descrição do problema: No celular a página aparecia minúscula (era preciso dar zoom) e, mesmo com zoom, rolava para os lados (issue #04).
- Causa identificada: Havia duas causas independentes: (1) o head não tinha a meta tag viewport, então o navegador móvel simulava uma tela de cerca de 980 px e reduzia a página; (2) a classe .container tinha largura fixa de 960 px, maior que a tela de um celular comum (360 px).
- Correção realizada: Adicionada a meta viewport (width=device-width, initial-scale=1) e a largura do .container passou a ser fluida: min(100% - 2rem, var(--largura-max)), com margem lateral de 1rem de cada lado.
- Arquivos alterados:
    - index.html
    - css/style.css

BUG-05

- Descrição do problema: O Google Chrome perguntava se o usuário queria traduzir a página do inglês e o leitor de tela lia os textos em português com sotaque americano (issue #05).

- Causa identificada: O elemento <html> declarava lang="en", informando ao navegador e às tecnologias assistivas que o conteúdo estava em inglês.
- Correção realizada: Atributo alterado para lang="pt-BR", igual ao que já era usado em resultado.html.
- Arquivos alterados:
    - index.html

BUG-06

- Descrição do problema: A página inicial tinha erros de ortografia: "Mapeamento Vocasional" e "interêsses" (que aparecia em dois lugares) (issue #06).
- Causa identificada: Erros de digitação nos textos do index.html, que não haviam passado por revisão.
- Correção realizada: Revisão de todos os textos da página: "Vocacional", "interesses" no passo 1 e "Áreas de interesse" no rótulo do campo.
- Arquivos alterados:
    - index.html

BUG-07

- Descrição do problema: Os rótulos "Nome", "Hobbies" e "Áreas de interesse" estavam quase brancos sobre o fundo branco do cartão e sumiam, principalmente com sol na tela do celular (issue #07).
- Causa identificada: A regra .campo label usava a cor #cfd3e3, cuja razão de contraste sobre #ffffff é de apenas 1,49:1, muito abaixo do mínimo de 4,5:1 exigido pela WCAG (nível AA) para texto.
- Correção realizada: A cor passou a usar a variável --cor-tinta (#1b2140), que já fazia parte da paleta do projeto. A nova razão de contraste é de 15,69:1.
- Arquivos alterados:
    - css/style.css

BUG-08

- Descrição do problema: Clicar no nome do campo (por exemplo, "Hobbies") não levava o cursor à caixa de texto e, ao navegar com Tab, não havia nenhum destaque indicando o elemento selecionado (issue #08).
- Causa identificada: (1) Os <label> não tinham o atributo for, portanto não estavam associados aos campos. (2) O CSS tinha a regra global *:focus { outline: none; }, que removia o contorno de foco de todos os elementos.
- Correção realizada: Cada <label> recebeu for apontando para o id do campo (as dicas também foram ligadas aos campos com aria-describedby). A regra

que removia o contorno foi apagada e foi criada a regra :focus-visible com contorno de 3 px na cor primária.
- Arquivos alterados:
    - index.html
    - css/style.css

BUG-09

- Descrição do problema: O Lighthouse apontava que a página não tinha título principal (h1), regiões de cabeçalho, conteúdo principal e rodapé, e que a lista de passos não era reconhecida como lista (issue #09).
- Causa identificada: A página inteira estava montada com elementos div, que não carregam significado para leitores de tela.
- Correção realizada: Os div foram trocados por header, main, section, footer, h1, h2, p e pela lista ordenada ol/li dos passos. Todas as classes CSS foram mantidas, de modo que o visual continua o mesmo.
- Arquivos alterados:
    - index.html

BUG-10

- Descrição do problema: O formulário aceitava hobbies em branco ou só com vírgulas (", , ,"), gerando no console a lista ["", " ", " ", ""] (issue #10).
- Causa identificada: O app.js usava split(",") sem remover espaços nem descartar itens vazios. O teste hobbies.length === 0 nunca era verdadeiro, porque split sempre devolve ao menos um item. Além disso, os campos não tinham required.
- Correção realizada: Os três campos receberam required. No app.js foi criada separarLista (split, trim e filter) e a validação com setCustomValidity e reportValidity, que exibe o aviso no próprio campo quando não sobra nenhum item.
- Arquivos alterados:
    - index.html
    - js/app.js

BUG-11

- Descrição do problema: Ao clicar no botão aparecia a caixa cinza padrão do navegador com o texto "Tente mais tarde" e o endereço do arquivo, o que parecia um golpe (issue #11).
- Causa identificada: O aviso era exibido com alert(), cuja aparência é controlada pelo navegador e não pode ser estilizada.

- Correção realizada: Criado um <dialog> com a identidade do site (título "Tente mais tarde", mensagem e botão "Entendi"). O app.js o abre com showModal(), o que já garante o fechamento com a tecla Esc, e o botão usa <form method="dialog"> para fechar sem código extra.
- Arquivos alterados:
    - index.html
    - js/app.js

BUG-12

- Descrição do problema: O formulário não enviava os dados para o sistema de análise: o código tinha apenas um TODO e sempre exibia o aviso "Tente mais tarde" (issue #12).
- Causa identificada: O envio à API não havia sido implementado no app.js; não existiam requisição, estado de carregamento, limite de tempo nem tratamento de erros.
- Correção realizada: Implementado o envio com fetch (POST, JSON, no formato do contrato do README) para CONFIG.API_URL. Se o endereço estiver vazio, abre o modal "Tente mais tarde". Durante o envio o botão fica desabilitado e mostra "Analisando seu perfil...". Há limite de tempo (CONFIG.TEMPO_LIMITE_MS) e qualquer status fora de 2xx vira erro; em erro ou demora o modal informa o problema e o formulário não é apagado. Em caso de sucesso o resultado é guardado no sessionStorage e a pessoa vai para resultado.html. O endereço da API continua vazio no config.js, pois cada grupo recebe o seu da professora.
- Arquivos alterados:
    - js/app.js

BUG-13

- Descrição do problema: A página resultado.html (inclusive pelo link "Ver um resultado de exemplo") abria vazia, apenas com cabeçalho e rodapé (issue #13).
- Causa identificada: O arquivo js/resultado.js continha somente comentários TODO. Nenhum código lia os dados nem preenchia a página, e as seções #resultado e #sem-resultado permaneciam com o atributo hidden.
- Correção realizada: Implementado o resultado.js: lê os dados de DADOS_EXEMPLO (com ?exemplo na URL) ou do sessionStorage; valida; ordena as carreiras da maior para a menor compatibilidade; clona o <template> para cada carreira (título, percentual, barra, resumo, competências e roteiro numerado); destaca a primeira com o selo "Melhor combinação"; e, sem dados válidos, mostra a mensagem amigável com o

botão para voltar ao formulário. Os textos vindos da API são inseridos com textContent.

- Arquivos alterados:
    - js/resultado.js

4. Evidências

BUG-01

Antes da correção

index.html

```
<button type="button" class="botao">Descobrir minha carreira</button>
```

Após a correção

index.html

```
<button type="submit" class="botao">Descobrir minha carreira</button>
```

BUG-02

Antes da correção

index.html

```
<input type="number" id="nome" name="nome" placeholder="Ex.: Aline Santos">
```

Após a correção

index.html

```
<input type="text" id="nome" name="nome" placeholder="Ex.: Aline Santos"
  required minlength="2" maxlength="60">
```

js/app.js

```
const nomeLimpo = nome.value.trim();
exigir(nome, nomeLimpo.length >= 2, 'Informe um nome com pelo menos 2 letras.');
```

BUG-03

Antes da correção

css/style.css

```
.formulario {
display: flex;
flex-direction: row;
gap: 1.25rem;
}
```

Após a correção

css/style.css

```
.formulario {
display: flex;
flex-direction: column;
gap: 1.25rem;
}
```

BUG-04

Antes da correção

index.html

```
<meta charset="UTF-8">
<!-- não havia <meta name="viewport"> -->
```

css/style.css

```
.container {
width: 960px;
margin-inline: auto;
}
```

Após a correção

index.html

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

css/style.css

```
.container {
width: min(100% - 2rem, var(--largura-max));
margin-inline: auto;
}
```

BUG-05

Antes da correção

index.html

```
<html lang="en">
```

Após a correção

index.html

```
<html lang="pt-BR">
```

BUG-06

Antes da correção

index.html

```
<div class="subtitulo">Como funciona o Mapeamento Vocasional</div>
<div class="passo">Preencha seu nome, hobbies e interêsses.</div>
<label>Áreas de interêsse</label>
```

Após a correção

index.html

```
<h2 class="subtitulo">Como funciona o Mapeamento Vocacional</h2>
<li class="passo">Preencha seu nome, hobbies e interesses.</li>
<label for="interesses">Áreas de interesse</label>
```

BUG-07

Antes da correção

css/style.css

```
.campo label {
font-weight: 700;
color: #cfd3e3;
}
```

Após a correção

css/style.css

```
.campo label {
font-weight: 700;
color: var(--cor-tinta);
}
```

BUG-08

Antes da correção

index.html

```
<label>Hobbies</label>
```

css/style.css

```
*:focus {
outline: none;
}
```

Após a correção

index.html

```
<label for="hobbies">Hobbies</label>
```

css/style.css

```
:focus-visible {
outline: 3px solid var(--cor-primaria);
outline-offset: 2px;
}
```

BUG-09

Antes da correção

index.html

```
<div class="topo">
<div class="titulo-principal">Descubra carreiras em tecnologia que combinam com
você</div>
<div class="passos">
<div class="rodape">
```

Após a correção

index.html

```
<header class="topo">
<main class="container pagina-inicial">
<h1 class="titulo-principal">Descubra carreiras em tecnologia que combinam com você</h1>
<ol class="passos">
<footer class="rodape">
```

BUG-10

Antes da correção

js/app.js

```
const hobbies = campos.hobbies.value.split(',');
const interesses = campos.interesses.value.split(',');
if (hobbies.length === 0 || interesses.length === 0) {
alert('Preencha os hobbies e interesses.');
return;
}
```

Após a correção

js/app.js

```
const separarLista = (texto) =>
  texto
    .split(',')
    .map((item) => item.trim())
    .filter((item) => item !== '');
// ...
exigir(hobbies, listaHobbies.length > 0, 'Informe pelo menos um hobby.');
exigir(interesses, listaInteresses.length > 0, 'Informe pelo menos um interesse.');
if (!formulario.reportValidity()) {
  return;
```

```
}
```

BUG-11

Antes da correção

js/app.js

```
alert('Tente mais tarde');
```

Após a correção

index.html

```
<dialog id="modal-aviso" class="modal" aria-labelledby="modal-titulo">
  <h2 id="modal-titulo">Tente mais tarde</h2>
  <p id="modal-mensagem">O serviço de análise ainda não está disponível.</p>
  <form method="dialog" class="modal__acoes">
    <button class="botao">Entendi</button>
  </form>
</dialog>
```

js/app.js

```
const abrirModal = (mensagem) => {
  modalMensagem.textContent = mensagem;
  modal.showModal();
};
```

BUG-12

Antes da correção

js/app.js

```
// TODO: enviar o perfil para a API configurada em CONFIG.API_URL
// e levar o usuário para a página resultado.html.
alert('Tente mais tarde');
```

Após a correção

js/app.js

```
if (!CONFIG.API_URL) {
  abrirModal('O serviço de análise ainda não está disponível.');
  return;
}
// ...
botaoEnviar.disabled = true;
botaoEnviar.textContent = 'Analisando seu perfil...';
// ...
const resposta = await fetch(CONFIG.API_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(perfil),
  signal: AbortSignal.timeout(CONFIG.TEMPO_LIMITE_MS),
});
```

```
if (!resposta.ok) {
  throw new Error(`A API respondeu com status ${resposta.status}`);
}
// ...
sessionStorage.setItem(CONFIG.CHAVE_RESULTADO, JSON.stringify(resultado));
window.location.href = 'resultado.html';
```

BUG-13

Antes da correção

js/resultado.js

```
// TODO 1: descobrir de onde vêm os dados
// TODO 2: se não houver dados válidos, mostre a seção #sem-resultado
// TODO 3: se houver dados, preencha nome e síntese e, para cada carreira,
```

Após a correção

js/resultado.js

```
const dados = lerDados();
if (dadosValidos(dados)) {
  const carreiras = [...dados.recommended_careers].sort((a, b) => b.match_score -
a.match_score);
  document.querySelector('#nome-usuario').textContent = dados.user_name;
  document.querySelector('#sintese').textContent = dados.career_identity_statement;
  listaCarreiras.replaceChildren(
    ...carreiras.map((carreira, indice) => criarCartao(carreira, indice === 0)),
  );
  secaoResultado.hidden = false;
} else {
  secaoSemResultado.hidden = false;
}
```

5. Testes realizados

Os testes foram executados nas versões original e corrigida do sistema, em Chromium (modo headless, Playwright), em tela de 1280 px e, no BUG-04, em emulação de celular de 360 px. Como o endereço real da API é fornecido pela professora, as respostas da API foram simuladas por interceptação das requisições.

Teste executado	Resultado esperado	Resultado obtido
BUG-01: preencher os campos e clicar em "Descobrir minha carreira" (API_URL vazia).	O formulário é enviado e o aviso "Tente mais tarde" aparece.	Modal exibido ao clicar. Na versão original, nada acontecia.
BUG-02: digitar "Aline" no campo Nome; tentar	O texto é aceito; nome vazio ou com 1 letra é bloqueado.	Campo aceitou "Aline" (type="text"); vazio e 1 letra foram bloqueados

enviar com o nome vazio e com uma letra.		pela validação nativa. Antes: campo numérico, ficava vazio.
BUG-03: medir a posição e a largura dos campos em tela de 1280 px.	Nome, Hobbies, Interesses e botão empilhados, com a largura do cartão.	Campos empilhados na ordem correta, todos com 437 px (largura interna do cartão); botão com 57 px de altura. Antes: campos de 52 e 73 px, botão com 285 px de altura.
BUG-04: abrir a página em emulação de celular de 360 px.	Página sem zoom e sem rolagem horizontal.	Layout de 360 px, escala 1 e página com 360 px de largura. Antes: layout de 980 px com escala 0,37; só com o meta viewport a página ainda tinha 960 px (causa 2).
BUG-05: ler o atributo lang do documento.	lang="pt-BR".	lang="pt-BR". Antes: "en".
BUG-06: procurar "Vocasional" e "interêsse" no texto da página.	Nenhuma ocorrência; texto com "Vocacional" e "interesses".	Nenhuma ocorrência dos erros; "Vocacional" presente. Antes: 1 ocorrência de "Vocasional" e 2 de "interêsse".
BUG-07: calcular o contraste dos rótulos sobre o fundo do cartão.	Razão de contraste de pelo menos 4,5:1 (WCAG AA).	15,69:1. Antes: 1,49:1.
BUG-08: clicar no rótulo "Hobbies" e percorrer a página com a tecla Tab.	O cursor vai para o campo e todos os elementos mostram contorno ao receber foco.	Clique no rótulo focou o campo hobbies; as 6 paradas do Tab mostraram contorno sólido de 3 px. Antes: o clique não focava nada e nenhuma parada tinha contorno.
BUG-09: verificar a estrutura da página inicial.	Um h1, regiões header/main/footer e lista ol com 3 itens.	1 h1, header, main e footer presentes, ol com 3 li; 5 div restantes. Antes: 0 h1, sem regiões, sem lista e 29 div.
BUG-10: enviar hobbies ", , ," e, em outro teste, " dançar , cantar ,, ".	Primeiro caso bloqueado com aviso no campo; segundo enviado sem espaços nem itens vazios.	Aviso "Informe pelo menos um hobby." no campo e nenhum envio. Segundo caso enviou ["dançar","cantar"]. Antes:

		console mostrava ["", " ", " ", ""].
BUG-11: acionar o aviso com API_URL vazia; fechar com Esc e com o botão "Entendi".	Janela do site (modal), sem alert do navegador, que fecha com Esc e com o botão.	Modal com título "Tente mais tarde" e botão "Entendi"; nenhum alert nativo; fechou com Esc e com o botão. Antes: alert do navegador.
BUG-12: enviar o formulário com a API simulada, clicando 10 vezes (cliques e Enter) durante o envio.	Botão desabilitado com indicação de carregamento e uma única requisição.	Botão desabilitado com o texto "Analisando seu perfil..."; 1 única requisição POST após as 10 tentativas.
BUG-12: conferir o corpo e o cabeçalho da requisição e a resposta 200 da API simulada.	POST com Content-Type application/json e campos name, hobbies e interests; ida para resultado.html.	Corpo {"name":"Ana Maria","hobbies":["dançar","cantar"],"interests":["design de jogos"]}; navegou para resultado.html com "Perfil de Ana Maria Clara" e 4 cartões.
BUG-12: simular erro 500, queda de rede e demora maior que o limite de tempo.	Modal com mensagem clara; dados digitados mantidos; botão liberado.	Nos três casos o modal abriu (mensagem de demora específica para o limite de tempo), os campos mantiveram o texto e o botão voltou a ficar ativo.
BUG-13: abrir resultado.html?exemplo.	Perfil, síntese e carreiras da mais para a menos compatível, com melhor combinação em destaque.	"Perfil de Ana Maria Clara", síntese e 4 cartões em ordem (97%, 89%, 82%, 74%) com barra, resumo, 5 competências e 4 passos; selo "Melhor combinação" só no primeiro. Antes: página em branco.
BUG-13: abrir resultado.html sem dados, com JSON inválido guardado e com dados contendo HTML.	Mensagem amigável com botão para o formulário; HTML exibido como texto.	Mensagem "Nenhum perfil para mostrar" com o botão "Preencher formulário" nos dois primeiros casos; o HTML foi mostrado como texto e não foi executado. Sem erros de JavaScript no console.

6. Conclusão

Justifique os erros mais difíceis de identificar

Os erros mais difíceis de identificar foram os que tinham mais de uma causa ou que ficavam escondidos atrás de outro defeito.

BUG-04 (celular): tinha duas causas independentes. Corrigir só a meta viewport não bastava: nos testes com tela de 360 px, a versão com apenas o meta ainda gerava uma página de 960 px de largura, por causa da largura fixa do .container. Foi preciso corrigir as duas para eliminar o zoom e a rolagem lateral.

BUG-01 e BUG-10/BUG-11: como o botão nunca enviava o formulário (type="button"), ele escondia os outros dois defeitos. Só depois de forçar o envio por código foi possível reproduzir a lista com itens vazios descrita pelo cliente e o alert do navegador.

BUG-10: o teste hobbies.length === 0 do código original parecia correto, mas split(",") sempre devolve ao menos um item, então a condição nunca era verdadeira. Além disso, o atributo required não barra texto formado só por espaços ou vírgulas, o que exigiu uma validação adicional em JavaScript.

BUG-07 e BUG-08: não geram erro no console e só aparecem no uso real. O contorno de foco havia sido removido por uma regra global (*:focus { outline: none; }) e só ficava evidente ao navegar com a tecla Tab. O contraste dos rótulos só foi confirmado ao calcular a razão de contraste (1,49:1, contra o mínimo de 4,5:1 da WCAG).

Quais conhecimentos de Front-End foram necessários?

Foram necessários conhecimentos de: HTML5 semântico e acessível (header, main, section, footer, h1 e h2, lista ordenada, label com for, dialog e aria-describedby); validação nativa de formulários (required, minlength e maxlength) e a Constraint Validation API (setCustomValidity e reportValidity); CSS (Flexbox, box model, unidades relativas, função min(), :focus-visible e contraste de cores segundo a WCAG); responsividade (meta viewport e layout fluido); JavaScript ES6 (const e let, arrow functions, template strings, async/await, fetch, AbortSignal.timeout, try/catch/finally, manipulação do DOM com template e textContent, sessionStorage); consumo de API REST seguindo um contrato em JSON; e uso das ferramentas do navegador (F12, modo dispositivo e Lighthouse) para reproduzir e diagnosticar os problemas.

O que a equipe aprendeu durante a atividade?

A equipe aprendeu a reproduzir cada problema antes de corrigi-lo, a procurar a causa real em vez de tratar apenas o sintoma e a validar cada correção com um teste que falha na versão original e passa na versão corrigida.

Também ficou claro o valor de usar recursos nativos do HTML5 (required, label com for, dialog) antes de recorrer a JavaScript, de seguir o contrato da API à risca, de tratar os estados de erro (API vazia, erro e demora) sem apagar o que a pessoa digitou e de pensar em acessibilidade e em celular desde o início, e não como ajuste final.